

✓ YOLO (You Only Look Once) for Real-Time Object Detection

YOLO (You Only Look Once) is a deep learning-based object detection algorithm designed for **fast and real-time image processing**. Unlike traditional object detection methods that analyze different regions of an image separately, YOLO processes the **entire image in a single pass through a neural network**, making it extremely efficient.

In YOLO, the input image is **divided into a grid of cells** (for example, $S \times S$). Each grid cell is responsible for detecting objects whose **center lies inside that cell**. For every grid cell, the network predicts:

- **Bounding box coordinates** (position and size of the object)
- **Confidence score** indicating the probability that an object exists in the predicted box
- **Class probabilities** representing the category of the detected object

By combining these predictions, YOLO can **detect multiple objects and classify them simultaneously** in a single forward pass of the network.

Because the entire image is processed at once, YOLO captures **global contextual information**, which helps reduce false detections compared to methods that examine small regions independently.

Due to its **high speed and efficiency**, YOLO is widely used in applications such as:

- Autonomous driving
- Video surveillance
- Robotics
- Real-time video analysis
- Smart cameras

Is YOLO Better than CNN?

YOLO is not necessarily "better than CNN" because **YOLO itself is built using a Convolutional Neural Network (CNN)**. The two are not directly comparable. Instead:

two are not directly comparable. Instead,

- **CNN (Convolutional Neural Network)** is a general deep learning architecture used for image-related tasks such as classification, detection, and segmentation.
- **YOLO (You Only Look Once)** is a specific **object detection algorithm that uses CNN as its backbone**.

Key Differences

Feature	CNN (Basic)	YOLO
Purpose	Mainly image classification	Object detection and classification
Output	Class label	Bounding boxes + object classes
Speed	Moderate	Very fast (real-time detection)
Detection ability	Cannot locate objects by default	Detects multiple objects in one image

Example

- A **basic CNN** might predict:
"This image contains a dog."
- **YOLO** can predict:
"There is a dog at this location (bounding box) and a person at another location."

Why YOLO is Popular

YOLO is widely used because it can:

- Detect **multiple objects simultaneously**
- Process images **in real time**
- Perform detection using a **single forward pass of the CNN**

Conclusion

- **CNN** is the **underlying neural network architecture**.
- **YOLO** is an **object detection framework built on CNN** that performs detection and classification in a single step.

- YOLO is an **object detection framework built on CNN** that performs detection and classification in a single step.
- Each bounding box predicts objects inside and gives confidence score. It gives position and label.

Repository link: <https://github.com/AlexeyAB/darknet>

✓ Downloading and cloning Darknet Repository:

```
!git clone https://github.com/AlexeyAB/darknet
```

```
Cloning into 'darknet'...
remote: Enumerating objects: 15941, done.
remote: Counting objects: 100% (32/32), done.
remote: Compressing objects: 100% (24/24), done.
remote: Total 15941 (delta 14), reused 9 (delta 8), pack-reused 15909 (from 2)
Receiving objects: 100% (15941/15941), 14.51 MiB | 7.05 MiB/s, done.
Resolving deltas: 100% (10720/10720), done.
```

```
ls
```

```
darknet/  sample_data/
```

```
cd darknet/
```

```
/content/darknet
```

```
ls
```

3rdparty/	data/	net_cam_v3.sh*
build/	docker-compose.yml	net_cam_v4.sh*
CCM/	Dockerfile.cpu	package.xml
cfg/	Dockerfile.gpu	README.md
cmake/	image_yolov3.sh*	scripts/
CMakeLists.txt	image_yolov4.sh*	src/
DarknetConfig.cmake.in	include/	vcpkg.json
darknet_images.nv	ison mined streams.sh*	video yolov3.sh*

darknet_images.py
darknet.py
darknet_video.py

json_image_extraction
LICENSE
Makefile

video_yolov4.sh*

✓ Compile the library

!make

```
mkdir -p ./obj/
mkdir -p backup
mkdir -p results
chmod +x *.sh
g++ -std=c++11 -std=c++11 -Iinclude/ -I3rdparty/stb/include -Wall -Wfatal-errors -Wno-unused-result -Wno-ur
g++ -std=c++11 -std=c++11 -Iinclude/ -I3rdparty/stb/include -Wall -Wfatal-errors -Wno-unused-result -Wno-ur
./src/http_stream.cpp: In member function 'bool JSON_sender::write(const char*)':
./src/http_stream.cpp:253:21: warning: unused variable 'n' [-Wunused-variable]
 253 |             int n = _write(client, outputbuf, outlen);
      |             ^
./src/http_stream.cpp: In function 'void set_track_id(detection*, int, float, float, float, int, int, int)'
./src/http_stream.cpp:866:27: warning: comparison of integer expressions of different signedness: 'int' and
 866 |         for (int i = 0; i < v.size(); ++i) {
      |                        ~^~~~~~
./src/http_stream.cpp:874:33: warning: comparison of integer expressions of different signedness: 'int' and
 874 |         for (int old_id = 0; old_id < old_dets.size(); ++old_id) {
      |                        ~~~~~^~~~~~
./src/http_stream.cpp:893:31: warning: comparison of integer expressions of different signedness: 'int' and
 893 |         for (int index = 0; index < new_dets_num*old_dets.size(); ++index) {
      |                        ~~~~~^~~~~~
./src/http_stream.cpp:929:28: warning: comparison of integer expressions of different signedness: 'std::dec
 929 |         if (old_dets_dq.size() > deque_size) old_dets_dq.pop_front();
      |                        ~~~~~~^~~~~~
gcc -Iinclude/ -I3rdparty/stb/include -Wall -Wfatal-errors -Wno-unused-result -Wno-unknown-pragmas -fPIC -r
./src/gemm.c: In function 'convolution_2d':
./src/gemm.c:2042:15: warning: unused variable 'out_w' [-Wunused-variable]
 2042 |         const int out_w = (w + 2 * pad - ksize) / stride + 1;    // output_width=input_width for stride
      |         ^~~~~
./src/gemm.c:2041:15: warning: unused variable 'out_h' [-Wunused-variable]
 2041 |         const int out_h = (h + 2 * pad - ksize) / stride + 1;    // output_height=input_height for stri
```

```

~~~~~
gcc -Iinclude/ -I3rdparty/stb/include -Wall -Wfatal-errors -Wno-unused-result -Wno-unknown-pragmas -fPIC -r
./src/utils.c: In function 'custom_hash':
./src/utils.c:1095:12: warning: suggest parentheses around assignment used as truth value [-Wparentheses]
1095 |         while (c = *str++)
      |                ^
In file included from /usr/include/string.h:535,
                from include/darknet.h:14,
                from ./src/utils.h:3,
                from ./src/utils.c:4:
In function 'strncpy',
    inlined from 'copy_string' at ./src/utils.c:565:5:
/usr/include/x86_64-linux-gnu/bits/string_fortified.h:95:10: warning: '__builtin_strncpy' specified bound c
95 |     return __builtin___strncpy_chk (__dest, __src, __len,
      |                ^~~~~~
196 |                                     __glibc_objsize (__dest));
      |                                     ~~~~~
./src/utils.c: In function 'copy_string':
./src/utils.c:565:22: note: length computed here
565 |     strncpy(copy, s, strlen(s)+1);
      |                ^~~~~~
gcc -Iinclude/ -I3rdparty/stb/include -Wall -Wfatal-errors -Wno-unused-result -Wno-unknown-pragmas -fPIC -r
gcc -Iinclude/ -I3rdparty/stb/include -Wall -Wfatal-errors -Wno-unused-result -Wno-unknown-pragmas -fPIC -r
./src/convolutional_layer.c: In function 'resize_convolutional_layer':
./src/convolutional_layer.c:891:9: warning: unused variable 'old_h' [-Wunused-variable]
891 |     int old_h = l->h;
      |         ^~~~~
./src/convolutional_layer.c:890:9: warning: unused variable 'old_w' [-Wunused-variable]

```

Start coding or [generate](#) with AI.

✓ Download the pre trained weights

```
!wget https://github.com/AlexeyAB/darknet/releases/download/darknet_yolo_v3_optimal/yolov4.weights
```

```

--2026-03-14 08:13:47--  https://github.com/AlexeyAB/darknet/releases/download/darknet_yolo_v3_optimal/yolo
Resolving github.com (github.com)... 20.205.243.166
Connecting to github.com (github.com)|20.205.243.166|:443... connected.

```

```
2026-03-14 08:13:54 (35.8 MB/s) - 'yolov4.weights' saved [257717640/257717640]
```

```
3rdparty/  
backup/  
build/  
CCM/  
cfg/  
cmake/  
CMakeLists.txt  
darknet*  
DarknetConfig.cmake.in  
darknet_images.py  
darknet.py  
darknet_video.py  
  
data/  
docker-compose.yml  
Dockerfile.cpu  
Dockerfile.gpu  
image_yolov3.sh*  
image_yolov4.sh*  
include/  
json_mjpeg_streams.sh*  
LICENSE  
Makefile  
net_cam_v3.sh*  
net_cam_v4.sh*  
  
obj/  
package.xml  
README.md  
results/  
scripts/  
src/  
vcpkg.json  
video_yolov3.sh*  
video_yolov4.sh*  
yolov4.weights
```

```
GPU isn't used
OpenCV isn't used - data augmentation will be slow
mini_batch = 1, batch = 8, time_steps = 1, train = 0
```

layer	filters	size/strd(dil)	input	output
0 conv	32	3 x 3/ 1	608 x 608 x 3 ->	608 x 608 x 32 0.639 BF
1 conv	64	3 x 3/ 2	608 x 608 x 32 ->	304 x 304 x 64 3.407 BF
2 conv	64	1 x 1/ 1	304 x 304 x 64 ->	304 x 304 x 64 0.757 BF

```

3 route 1 -> 304 x 304 x 64
4 conv 64 1 x 1/ 1 304 x 304 x 64 -> 304 x 304 x 64 0.757 BF
5 conv 32 1 x 1/ 1 304 x 304 x 64 -> 304 x 304 x 32 0.379 BF
6 conv 64 3 x 3/ 1 304 x 304 x 32 -> 304 x 304 x 64 3.407 BF
7 Shortcut Layer: 4, wt = 0, wn = 0, outputs: 304 x 304 x 64 0.006 BF
8 conv 64 1 x 1/ 1 304 x 304 x 64 -> 304 x 304 x 64 0.757 BF
9 route 8 2 -> 304 x 304 x 128
10 conv 64 1 x 1/ 1 304 x 304 x 128 -> 304 x 304 x 64 1.514 BF
11 conv 128 3 x 3/ 2 304 x 304 x 64 -> 152 x 152 x 128 3.407 BF
12 conv 64 1 x 1/ 1 152 x 152 x 128 -> 152 x 152 x 64 0.379 BF
13 route 11 -> 152 x 152 x 128
14 conv 64 1 x 1/ 1 152 x 152 x 128 -> 152 x 152 x 64 0.379 BF
15 conv 64 1 x 1/ 1 152 x 152 x 64 -> 152 x 152 x 64 0.189 BF
16 conv 64 3 x 3/ 1 152 x 152 x 64 -> 152 x 152 x 64 1.703 BF
17 Shortcut Layer: 14, wt = 0, wn = 0, outputs: 152 x 152 x 64 0.001 BF
18 conv 64 1 x 1/ 1 152 x 152 x 64 -> 152 x 152 x 64 0.189 BF
19 conv 64 3 x 3/ 1 152 x 152 x 64 -> 152 x 152 x 64 1.703 BF
20 Shortcut Layer: 17, wt = 0, wn = 0, outputs: 152 x 152 x 64 0.001 BF
21 conv 64 1 x 1/ 1 152 x 152 x 64 -> 152 x 152 x 64 0.189 BF
22 route 21 12 -> 152 x 152 x 128
23 conv 128 1 x 1/ 1 152 x 152 x 128 -> 152 x 152 x 128 0.757 BF
24 conv 256 3 x 3/ 2 152 x 152 x 128 -> 76 x 76 x 256 3.407 BF
25 conv 128 1 x 1/ 1 76 x 76 x 256 -> 76 x 76 x 128 0.379 BF
26 route 24 -> 76 x 76 x 256
27 conv 128 1 x 1/ 1 76 x 76 x 256 -> 76 x 76 x 128 0.379 BF
28 conv 128 1 x 1/ 1 76 x 76 x 128 -> 76 x 76 x 128 0.189 BF
29 conv 128 3 x 3/ 1 76 x 76 x 128 -> 76 x 76 x 128 1.703 BF
30 Shortcut Layer: 27, wt = 0, wn = 0, outputs: 76 x 76 x 128 0.001 BF
31 conv 128 1 x 1/ 1 76 x 76 x 128 -> 76 x 76 x 128 0.189 BF
32 conv 128 3 x 3/ 1 76 x 76 x 128 -> 76 x 76 x 128 1.703 BF
33 Shortcut Layer: 30, wt = 0, wn = 0, outputs: 76 x 76 x 128 0.001 BF
34 conv 128 1 x 1/ 1 76 x 76 x 128 -> 76 x 76 x 128 0.189 BF
35 conv 128 3 x 3/ 1 76 x 76 x 128 -> 76 x 76 x 128 1.703 BF
36 Shortcut Layer: 33, wt = 0, wn = 0, outputs: 76 x 76 x 128 0.001 BF
37 conv 128 1 x 1/ 1 76 x 76 x 128 -> 76 x 76 x 128 0.189 BF
38 conv 128 3 x 3/ 1 76 x 76 x 128 -> 76 x 76 x 128 1.703 BF
39 Shortcut Layer: 36, wt = 0, wn = 0, outputs: 76 x 76 x 128 0.001 BF
40 conv 128 1 x 1/ 1 76 x 76 x 128 -> 76 x 76 x 128 0.189 BF
41 conv 128 3 x 3/ 1 76 x 76 x 128 -> 76 x 76 x 128 1.703 BF
42 Shortcut Layer: 39, wt = 0, wn = 0, outputs: 76 x 76 x 128 0.001 BF
43 conv 128 1 x 1/ 1 76 x 76 x 128 -> 76 x 76 x 128 0.189 BF

```



```

43 conv 128 1 x 1/ 1 76 x 76 x 128 -> 76 x 76 x 128 0.189 BF
44 conv 128 3 x 3/ 1 76 x 76 x 128 -> 76 x 76 x 128 1.703 BF
45 Shortcut Layer: 42, wt = 0, wn = 0, outputs: 76 x 76 x 128 0.001 BF
46 conv 128 1 x 1/ 1 76 x 76 x 128 -> 76 x 76 x 128 0.189 BF
47 conv 128 3 x 3/ 1 76 x 76 x 128 -> 76 x 76 x 128 1.703 BF
48 Shortcut Layer: 45, wt = 0, wn = 0, outputs: 76 x 76 x 128 0.001 BF
49 conv 128 1 x 1/ 1 76 x 76 x 128 -> 76 x 76 x 128 0.189 BF
50 conv 128 3 x 3/ 1 76 x 76 x 128 -> 76 x 76 x 128 1.703 BF
51 Shortcut Layer: 48, wt = 0, wn = 0, outputs: 76 x 76 x 128 0.001 BF
52 conv 128 1 x 1/ 1 76 x 76 x 128 -> 76 x 76 x 128 0.189 BF
53 route 52 25 -> 76 x 76 x 256

```

```

import cv2
import matplotlib.pyplot as plt

image = cv2.imread('predictions.jpg')

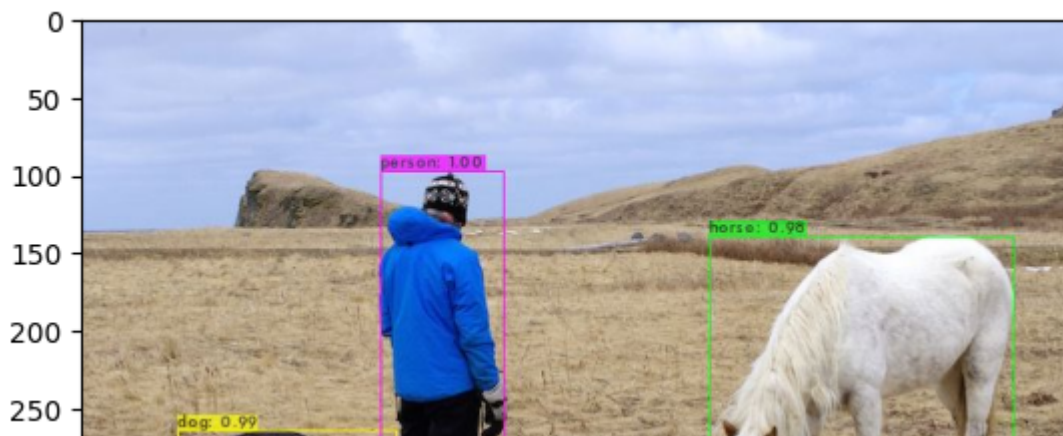
```

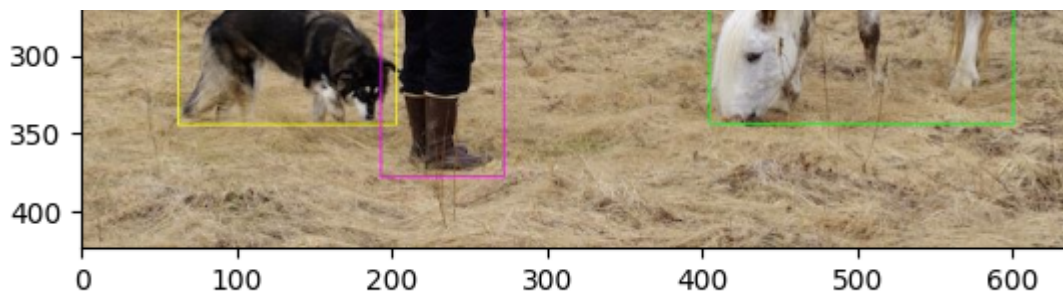
```

def show_detection(path):
    image = cv2.imread(path)
    plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))

```

```
show_detection('predictions.jpg')
```





✓ Darknet and GPU

```
import tensorflow as tf
tf.test.gpu_device_name()
```

```
 '/device:GPU:0'
```

```
ls
```

```
3rdparty/      data/          obj/
backup/        docker-compose.yml  package.xml
build/         Dockerfile.cpu    predictions.jpg
CCM/          Dockerfile.gpu    README.md
cfg/           image_yolov3.sh*  results/
cmake/         image_yolov4.sh*  scripts/
CMakeLists.txt include/          src/
darknet*       json_mjpeg_streams.sh*  vcpkg.json
DarknetConfig.cmake.in LICENSE          video_yolov3.sh*
darknet_images.py Makefile        video_yolov4.sh*
darknet.py     net_cam_v3.sh*    yolov4.weights
darknet_video.py net_cam_v4.sh*
```

Line	Purpose
OPENCV=1	Enables OpenCV for image/video processing
GPU=1	Enables GPU acceleration
CUDA=1	Enables CUDA support for GPU acceleration

CUDNN=1 Enables NVIDIA cuDNN for faster deep learning

```
# Enable OpenCV
!sed -i 's/OPENCV=0/OPENCV=1/' Makefile

# Enable GPU
!sed -i 's/GPU=0/GPU=1/' Makefile

# Enable cuDNN
!sed -i 's/CUDNN=0/CUDNN=1/' Makefile
```

```
#!/make
```

```
!./darknet detect cfg/yolov4.cfg yolov4.weights data/horses.jpg
```

```
GPU isn't used
OpenCV isn't used - data augmentation will be slow
mini_batch = 1, batch = 8, time_steps = 1, train = 0
layer  filters  size/strd(dil)    input                                output
0 conv     32          3 x 3/ 1         608 x 608 x   3 -> 608 x 608 x  32 0.639 BF
1 conv     64          3 x 3/ 2         608 x 608 x  32 -> 304 x 304 x  64 3.407 BF
2 conv     64          1 x 1/ 1         304 x 304 x  64 -> 304 x 304 x  64 0.757 BF
3 route    1
4 conv     64          1 x 1/ 1         304 x 304 x  64 -> 304 x 304 x  64 0.757 BF
5 conv     32          1 x 1/ 1         304 x 304 x  64 -> 304 x 304 x  32 0.379 BF
6 conv     64          3 x 3/ 1         304 x 304 x  32 -> 304 x 304 x  64 3.407 BF
7 Shortcut Layer: 4, wt = 0, wn = 0, outputs: 304 x 304 x  64 0.006 BF
8 conv     64          1 x 1/ 1         304 x 304 x  64 -> 304 x 304 x  64 0.757 BF
9 route    8 2
10 conv    64          1 x 1/ 1         304 x 304 x 128 -> 304 x 304 x  64 1.514 BF
11 conv    128         3 x 3/ 2         304 x 304 x  64 -> 152 x 152 x 128 3.407 BF
12 conv    64          1 x 1/ 1         152 x 152 x 128 -> 152 x 152 x  64 0.379 BF
13 route   11
14 conv    64          1 x 1/ 1         152 x 152 x 128 -> 152 x 152 x  64 0.379 BF
15 conv    64          1 x 1/ 1         152 x 152 x  64 -> 152 x 152 x  64 0.189 BF
16 conv    64          3 x 3/ 1         152 x 152 x  64 -> 152 x 152 x  64 1.703 BF
17 Shortcut Layer: 14, wt = 0, wn = 0, outputs: 152 x 152 x  64 0.001 BF
18 conv    64          1 x 1/ 1         152 x 152 x  64 -> 152 x 152 x  64 0.189 BF
```

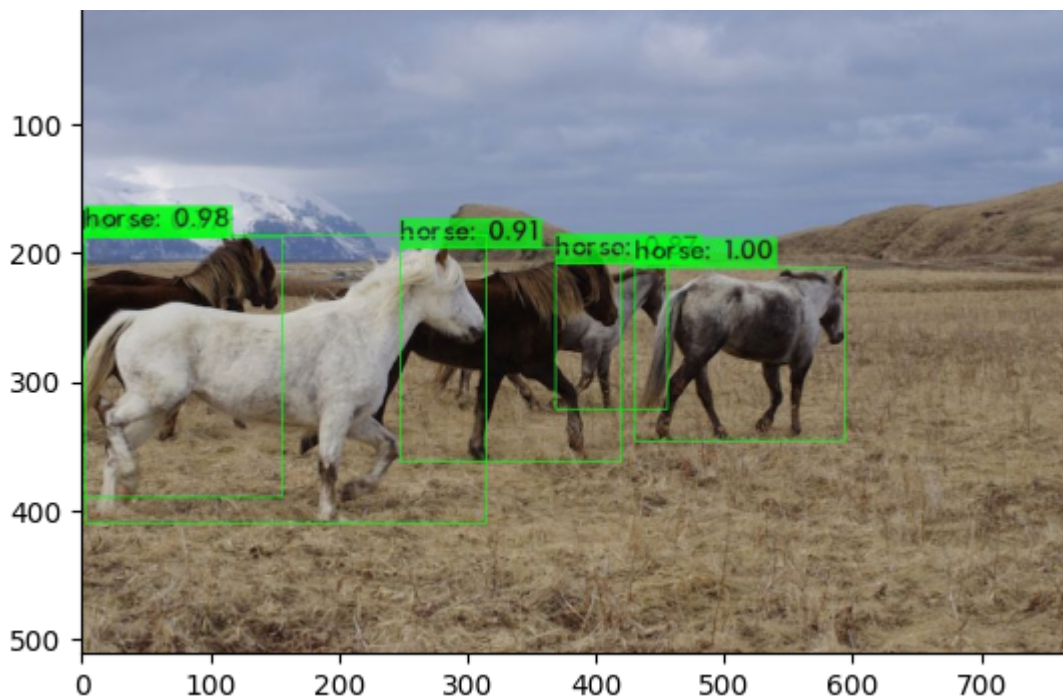
```

18 conv      64      1 x 1/ 1    152 x 152 x 64 -> 152 x 152 x 64 0.189 BF
19 conv      64      3 x 3/ 1    152 x 152 x 64 -> 152 x 152 x 64 1.703 BF
20 Shortcut Layer: 17, wt = 0, wn = 0, outputs: 152 x 152 x 64 0.001 BF
21 conv      64      1 x 1/ 1    152 x 152 x 64 -> 152 x 152 x 64 0.189 BF
22 route 21 12
23 conv     128      1 x 1/ 1    152 x 152 x 128 -> 152 x 152 x 128 0.757 BF
24 conv     256      3 x 3/ 2    152 x 152 x 128 -> 76 x 76 x 256 3.407 BF
25 conv     128      1 x 1/ 1     76 x 76 x 256 -> 76 x 76 x 128 0.379 BF
26 route 24
27 conv     128      1 x 1/ 1     76 x 76 x 256 -> 76 x 76 x 128 0.379 BF
28 conv     128      1 x 1/ 1     76 x 76 x 128 -> 76 x 76 x 128 0.189 BF
29 conv     128      3 x 3/ 1     76 x 76 x 128 -> 76 x 76 x 128 1.703 BF
30 Shortcut Layer: 27, wt = 0, wn = 0, outputs: 76 x 76 x 128 0.001 BF
31 conv     128      1 x 1/ 1     76 x 76 x 128 -> 76 x 76 x 128 0.189 BF
32 conv     128      3 x 3/ 1     76 x 76 x 128 -> 76 x 76 x 128 1.703 BF
33 Shortcut Layer: 30, wt = 0, wn = 0, outputs: 76 x 76 x 128 0.001 BF
34 conv     128      1 x 1/ 1     76 x 76 x 128 -> 76 x 76 x 128 0.189 BF
35 conv     128      3 x 3/ 1     76 x 76 x 128 -> 76 x 76 x 128 1.703 BF
36 Shortcut Layer: 33, wt = 0, wn = 0, outputs: 76 x 76 x 128 0.001 BF
37 conv     128      1 x 1/ 1     76 x 76 x 128 -> 76 x 76 x 128 0.189 BF
38 conv     128      3 x 3/ 1     76 x 76 x 128 -> 76 x 76 x 128 1.703 BF
39 Shortcut Layer: 36, wt = 0, wn = 0, outputs: 76 x 76 x 128 0.001 BF
40 conv     128      1 x 1/ 1     76 x 76 x 128 -> 76 x 76 x 128 0.189 BF
41 conv     128      3 x 3/ 1     76 x 76 x 128 -> 76 x 76 x 128 1.703 BF
42 Shortcut Layer: 39, wt = 0, wn = 0, outputs: 76 x 76 x 128 0.001 BF
43 conv     128      1 x 1/ 1     76 x 76 x 128 -> 76 x 76 x 128 0.189 BF
44 conv     128      3 x 3/ 1     76 x 76 x 128 -> 76 x 76 x 128 1.703 BF
45 Shortcut Layer: 42, wt = 0, wn = 0, outputs: 76 x 76 x 128 0.001 BF
46 conv     128      1 x 1/ 1     76 x 76 x 128 -> 76 x 76 x 128 0.189 BF
47 conv     128      3 x 3/ 1     76 x 76 x 128 -> 76 x 76 x 128 1.703 BF
48 Shortcut Layer: 45, wt = 0, wn = 0, outputs: 76 x 76 x 128 0.001 BF
49 conv     128      1 x 1/ 1     76 x 76 x 128 -> 76 x 76 x 128 0.189 BF
50 conv     128      3 x 3/ 1     76 x 76 x 128 -> 76 x 76 x 128 1.703 BF
51 Shortcut Layer: 48, wt = 0, wn = 0, outputs: 76 x 76 x 128 0.001 BF
52 conv     128      1 x 1/ 1     76 x 76 x 128 -> 76 x 76 x 128 0.189 BF
53 route 52 25

```

```
show_detection('predictions.jpg')
```

0



```
!./darknet detect cfg/yolov4.cfg yolov4.weights data/horses.jpg -thresh 0.9
```

```
GPU isn't used
OpenCV isn't used - data augmentation will be slow
mini_batch = 1, batch = 8, time_steps = 1, train = 0
```

layer	filters	size/strd(dil)	input	output
0 conv	32	3 x 3/ 1	608 x 608 x 3 ->	608 x 608 x 32 0.639 BF
1 conv	64	3 x 3/ 2	608 x 608 x 32 ->	304 x 304 x 64 3.407 BF
2 conv	64	1 x 1/ 1	304 x 304 x 64 ->	304 x 304 x 64 0.757 BF
3 route	1		->	304 x 304 x 64
4 conv	64	1 x 1/ 1	304 x 304 x 64 ->	304 x 304 x 64 0.757 BF
5 conv	32	1 x 1/ 1	304 x 304 x 64 ->	304 x 304 x 32 0.379 BF
6 conv	64	3 x 3/ 1	304 x 304 x 32 ->	304 x 304 x 64 3.407 BF
7 Shortcut	Layer: 4,	wt = 0, wn = 0,	outputs: 304 x 304 x 64	0.006 BF
8 conv	64	1 x 1/ 1	304 x 304 x 64 ->	304 x 304 x 64 0.757 BF
9 route	8 2		->	304 x 304 x 128
10 conv	64	1 x 1/ 1	304 x 304 x 128 ->	304 x 304 x 64 1.514 BF
11 conv	128	3 x 3/ 2	304 x 304 x 64 ->	152 x 152 x 128 3.407 BF

```

12 conv      64      1 x 1/ 1    152 x 152 x 128 -> 152 x 152 x 64 0.379 BF
13 route    11
14 conv      64      1 x 1/ 1    152 x 152 x 128 -> 152 x 152 x 64 0.379 BF
15 conv      64      1 x 1/ 1    152 x 152 x 64 -> 152 x 152 x 64 0.189 BF
16 conv      64      3 x 3/ 1    152 x 152 x 64 -> 152 x 152 x 64 1.703 BF
17 Shortcut Layer: 14, wt = 0, wn = 0, outputs: 152 x 152 x 64 0.001 BF
18 conv      64      1 x 1/ 1    152 x 152 x 64 -> 152 x 152 x 64 0.189 BF
19 conv      64      3 x 3/ 1    152 x 152 x 64 -> 152 x 152 x 64 1.703 BF
20 Shortcut Layer: 17, wt = 0, wn = 0, outputs: 152 x 152 x 64 0.001 BF
21 conv      64      1 x 1/ 1    152 x 152 x 64 -> 152 x 152 x 64 0.189 BF
22 route    21 12
23 conv     128      1 x 1/ 1    152 x 152 x 128 -> 152 x 152 x 128 0.757 BF
24 conv     256      3 x 3/ 2    152 x 152 x 128 -> 76 x 76 x 256 3.407 BF
25 conv     128      1 x 1/ 1     76 x 76 x 256 -> 76 x 76 x 128 0.379 BF
26 route    24
27 conv     128      1 x 1/ 1     76 x 76 x 256 -> 76 x 76 x 128 0.379 BF
28 conv     128      1 x 1/ 1     76 x 76 x 128 -> 76 x 76 x 128 0.189 BF
29 conv     128      3 x 3/ 1     76 x 76 x 128 -> 76 x 76 x 128 1.703 BF
30 Shortcut Layer: 27, wt = 0, wn = 0, outputs: 76 x 76 x 128 0.001 BF
31 conv     128      1 x 1/ 1     76 x 76 x 128 -> 76 x 76 x 128 0.189 BF
32 conv     128      3 x 3/ 1     76 x 76 x 128 -> 76 x 76 x 128 1.703 BF
33 Shortcut Layer: 30, wt = 0, wn = 0, outputs: 76 x 76 x 128 0.001 BF
34 conv     128      1 x 1/ 1     76 x 76 x 128 -> 76 x 76 x 128 0.189 BF
35 conv     128      3 x 3/ 1     76 x 76 x 128 -> 76 x 76 x 128 1.703 BF
36 Shortcut Layer: 33, wt = 0, wn = 0, outputs: 76 x 76 x 128 0.001 BF
37 conv     128      1 x 1/ 1     76 x 76 x 128 -> 76 x 76 x 128 0.189 BF
38 conv     128      3 x 3/ 1     76 x 76 x 128 -> 76 x 76 x 128 1.703 BF
39 Shortcut Layer: 36, wt = 0, wn = 0, outputs: 76 x 76 x 128 0.001 BF
40 conv     128      1 x 1/ 1     76 x 76 x 128 -> 76 x 76 x 128 0.189 BF
41 conv     128      3 x 3/ 1     76 x 76 x 128 -> 76 x 76 x 128 1.703 BF
42 Shortcut Layer: 39, wt = 0, wn = 0, outputs: 76 x 76 x 128 0.001 BF
43 conv     128      1 x 1/ 1     76 x 76 x 128 -> 76 x 76 x 128 0.189 BF
44 conv     128      3 x 3/ 1     76 x 76 x 128 -> 76 x 76 x 128 1.703 BF
45 Shortcut Layer: 42, wt = 0, wn = 0, outputs: 76 x 76 x 128 0.001 BF
46 conv     128      1 x 1/ 1     76 x 76 x 128 -> 76 x 76 x 128 0.189 BF
47 conv     128      3 x 3/ 1     76 x 76 x 128 -> 76 x 76 x 128 1.703 BF
48 Shortcut Layer: 45, wt = 0, wn = 0, outputs: 76 x 76 x 128 0.001 BF
49 conv     128      1 x 1/ 1     76 x 76 x 128 -> 76 x 76 x 128 0.189 BF
50 conv     128      3 x 3/ 1     76 x 76 x 128 -> 76 x 76 x 128 1.703 BF
51 Shortcut Layer: 48, wt = 0, wn = 0, outputs: 76 x 76 x 128 0.001 BF

```

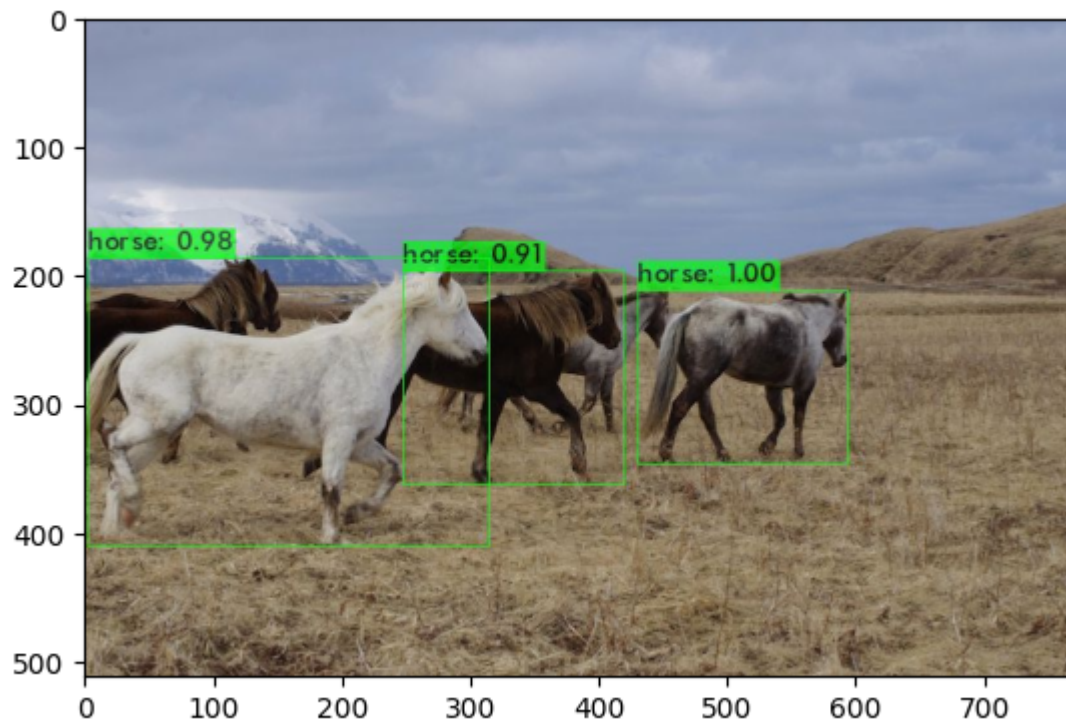


```

52 conv      128          1 x 1/ 1      76 x  76 x 128 ->  76 x  76 x 128 0.189 BF
53 route    52 25                                ->  76 x  76 x 256

```

```
show_detection('predictions.jpg')
```



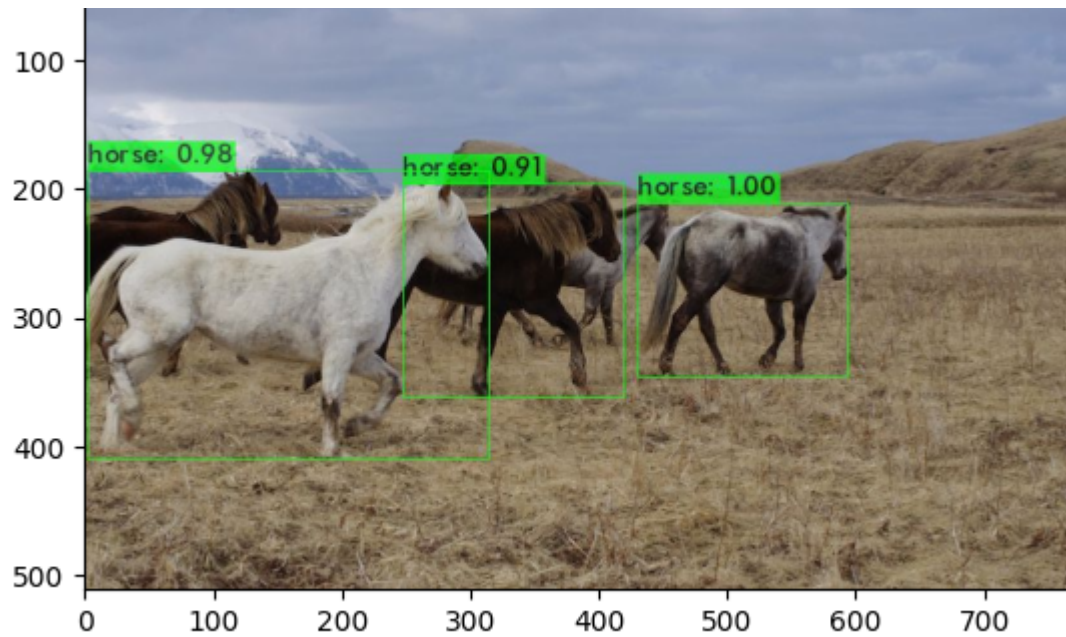
```

def show_detection(path):
    image = cv2.imread(path)
    plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
    plt.savefig('/content/drive/MyDrive/MCV_Continuatin_inGoogle_Colab/13_YOLO/output.jpg')

```

```
show_detection('predictions.jpg')
```





Start coding or [generate](#) with AI.

